

Actividad 0-ESTRUCTURAS DE DATOS. ANÁLISIS Y DESARROLLO DE SOFTWARE

Thomas Isaza Chalarca

Liney Patricia Ricardo

Sara Velásquez

Instructor: Luis Fernando

Base datos SQL(Relacionales)

Institución: CTMA Sena

Antioquia

2025

Guía Actividad 0 – Conceptos Generales: Estructuras de Datos

ANÁLISIS Y DESARROLLO DE SOFTWARE

CENTRO DE TECNOLOGÍA DE LA MANUFACTURA AVANZADA
MEDELLÍN



THOMAS ISAZA CHALARCA FICHA: 3169892

Palabras claves (key words)

Dato, Estructura, Bit, Sistema Numérico, Almacenamiento de información.

Responde y Dialoga con tu grupo:

- ¿Qué es un dato?
Un dato es una pieza elemental, un símbolo o conjunto de símbolos que representan una magnitud, una característica o un hecho. Es la unidad básica de información que, cuando se organiza, permite construir conocimientos. Ejemplos de datos pueden ser números, palabras, fechas o cualquier símbolo que tenga un significado en un contexto específico.
- ¿Qué es información?
La información es el conjunto de datos procesados, organizados y contextualizados que tienen significado y utilidad para quienes los reciben. Es el resultado de interpretar y organizar los datos de manera que puedan ser analizados para tomar decisiones o entender una situación. Por ejemplo, la lista de ventas diaria se convierte en información cuando se analiza para identificar tendencias.
- ¿Cómo se puede garantizar la disponibilidad de los datos en la empresa?
Se puede garantizar a través de estrategias como realizar copias de seguridad periódicas, implementar sistemas de almacenamiento confiables (como bases de datos que sean accesibles y protegidas), tener controles de acceso adecuados, usar redundancia en la infraestructura técnica, y establecer políticas de recuperación ante fallas o accidentes. La clave es que los datos estén accesibles cuando se necesiten, sin pérdida o interrupciones.
- ¿A qué nos referimos con confiabilidad de la información?
La confiabilidad de la información implica que los datos sean precisos, completos, coherentes y libres de errores. Esto asegura que las decisiones que se tomen basándose en

esa información sean correctas y efectivas. Una información confiable aumenta la confianza en los procesos y resultados de la organización y evita errores por datos incorrectos o incompletos.

- ¿Por qué consideras que la integridad de la información es esencial?
Porque garantiza que los datos no hayan sido alterados, dañados o manipulados de manera no autorizada. La integridad asegura que la información refleje cómo debe ser la realidad y que su contenido sea auténtico y correcto desde su origen hasta su uso final. Esto es vital para la toma de decisiones acertadas, el cumplimiento normativo y la confianza entre los usuarios y la organización.
- ¿Qué tanto puede impactar una falla de autenticidad en los datos de una organización?
Puede tener consecuencias graves, como decisiones basadas en información falsa, pérdida de confianza de los clientes, sanciones legales, errores en procesos críticos y daños a la reputación. La autenticidad asegura que la información proviene de fuentes confiables y no ha sido alterada maliciosamente, por lo que cualquier falla en esta área puede comprometer la integridad y la seguridad de toda la organización.

Elabore una tabla en la cual simbolice las magnitudes siguientes en sistema numérico decimal y su correspondencia en los sistemas numéricos descritos:

- 5050 • 680 • 780 • 4096 • 512 • 912 • 4567 •
 300 • 360 • 3800 • 256 • 556 • 1098 • 128 •
 428
- 1024 • 190 • 990
- 2048 • 64 • 604 • 2512 • 56 • 506
- 1970 • 10 • 100

Magnitud decimal	Binario	Octal	Hexadecimal
5050	1001110111010	11672	13BA
4096	1000000000000	10000	1000
4567	1000111001111	10727	11D7
3800	111011011000	7330	E8D
1098	10001001110	2112	44A
1024	10000000000	2000	400
2048	100000000000	4000	800
2512	1001110011000	4720	9D0
1970	11110110010	3662	7B2
680	1010101000	1250	2A8
512	1000000000	1000	200
300	100101100	454	12C
256	100000000	400	100
128	10000000	200	80
190	10111110	276	BE

64	1000000	100	40
56	111000	70	38
10	1010	12	A
780	1100001100	1414	30C
912	1110010000	1620	390
360	101101000	550	168
556	10001001100	1054	22C
428	110101100	654	1AC
990	1111011010	1736	3DE
604	1001011100	1134	25C
506	111111010	772	1FA
100	1100100	144	64

Consulta:

¿Qué operaciones aritméticas pueden ejecutarse con los sistemas numéricos?

Operaciones aritméticas en sistemas numéricos:

Binario: suma, resta, multiplicación y división con reglas de acarreo y préstamo propias del binario.

Octal y hexadecimal: también soportan las mismas operaciones, aunque a veces requieren conversión previa o uso de reglas específicas debido a la base mayor.

Las operaciones aritméticas que pueden realizarse en los diferentes sistemas numéricos incluyen la suma, resta, multiplicación y división, similares a las que se efectúan en el sistema decimal. Sin embargo, la ejecución de estas operaciones en sistemas binario, octal o hexadecimal requiere seguir reglas específicas para cada base, especialmente en cuanto a manejo de acarreo o préstamos en la suma y resta, y las multiplicaciones y divisiones.

¿Cómo se hacen conversiones de un sistema a otro: binario a octal, octal a decimal, decimal a hexadecimal, ¿etc.? **Conversiones entre sistemas:**

Binario a octal: Agrupar los bits en tríos desde la derecha y convertir cada grupo a su equivalente decimal.

Octal a decimal: Multiplicar cada dígito por 8 elevado a su posición (de derecha a izquierda), sumar los resultados.

Decimal a hexadecimal: Dividir entre 16 repetidamente, tomar los residuos y convertirlos a dígitos hexadecimales.

Otros ejemplos: Para convertir de hexadecimal a decimal, se multiplica cada dígito por 16 elevado a su posición; y de hexadecimal a binario, se reemplazan cada dígito por su equivalente en 4 bits.

¿Qué son: ¿bit, nibble y byte?

Bit: Unidad básica de información en computación, representa un valor binario 0 o 1.

Nibble: Conjunto de 4 bits, permite representar valores del 0 al 15 en decimal.

Byte: Conjunto de 8 bits, utilizado para representar un carácter en muchas codificaciones, con valores del 0 al 255 en decimal.

Actividad 2: Almacenamiento de Datos

¿Cómo utilizar la memoria y el tiempo de ejecución eficientemente con estructuras de datos apropiadas según los requerimientos y recursos disponibles al momento de desarrollar una aplicación?

Para desarrollar una aplicación eficiente, es esencial elegir estructuras de datos apropiadas según los requerimientos funcionales y los recursos disponibles del sistema. Esto implica analizar si se prioriza la velocidad o el ahorro de memoria. Por ejemplo, si se necesita acceso rápido a elementos, una tabla hash (HashMap) es ideal, mientras que, si se desea bajo consumo de memoria con datos simples, es mejor usar arreglos (arrays) o listas.

Cada estructura de datos tiene ventajas y desventajas: las listas enlazadas permiten inserciones y eliminaciones rápidas, pero consumen más memoria; los árboles balanceados ofrecen búsquedas ordenadas eficientes; las colas y pilas son útiles en flujos de datos o algoritmos. La complejidad temporal (como $O(1)$, $O(\log n)$, $O(n)$) también guía la elección. En sistemas con poca memoria, deben preferirse estructuras compactas y evitar la duplicación de datos. En cambio, si se busca rendimiento, es recomendable sacrificar algo de memoria para obtener mayor velocidad. Además, el uso de algoritmos adecuados y herramientas como cachés, profilers o técnicas como lazy loading permite optimizar aún más el rendimiento del sistema.

En resumen, una buena elección de estructura de datos, acompañada de un análisis de recursos y necesidades, es clave para construir aplicaciones eficientes y escalables.

Consulte:

¿Cuál es la estructura de un registro de memoria?

Un **registro de memoria** (más comúnmente referido como *registro del procesador*) es una unidad de almacenamiento pequeña y rápida ubicada dentro del procesador (CPU).

Estructura general de un registro:

- **Nombre o Identificador:** como AX, BX, R1, R2, etc.
- **Tamaño:** depende de la arquitectura (por ejemplo, 8, 16, 32 o 64 bits).

- **Propósito:** puede ser general (almacenamiento temporal) o especializado (contador de programa, puntero de pila, etc.).

Tipos de registros comunes:

- **Registros de propósito general:** para cálculos y almacenamiento temporal.
- **Registros de dirección:** contienen direcciones de memoria.
- **Registros de control:** como el contador de programa (PC) y registro de estado (FLAGS).
- **Registros de datos:** para almacenar datos inmediatos.

2. ¿Cómo se crea un puntero en los lenguajes de programación?

Un puntero es una variable que almacena la dirección de memoria de otra variable.

Ejemplo en C/C++: int

numero = 10;

int *puntero = № // puntero almacena la dirección de 'numero'

- int *puntero; → se declara un puntero a entero.
- № → operador que obtiene la dirección de memoria de número.

En otros lenguajes:

- En **Python** y **Java**, los punteros no son accesibles directamente, pero los objetos funcionan como referencias a memoria.

3. ¿Qué es un sistema de archivos y cuáles son los más utilizados?

Un sistema de archivos es el método que usa un sistema operativo para organizar y almacenar archivos en un dispositivo de almacenamiento (como disco duro o SSD).

Funciones:

- Asignar espacio a archivos.
- Llevar el control de la ubicación de los datos.
- Controlar acceso, permisos, y estructura de directorios.

Sistemas de archivos más utilizados:

Sistemas de archivos	Usado en
NTFS	Windows
FAT32 / exFAT	Discos portátiles, USB
Ext4	Linux
APFS	macOS (Apple File System)
HFS+	Sistemas macOS más antiguos

4. ¿Cómo se almacenan los datos en un disco duro?

Los datos en un disco duro (HDD) se almacenan en discos magnéticos llamados platos, los cuales giran a altas velocidades. **Proceso:**

1. Los platos están recubiertos con un material magnético.
2. Una cabeza lectora/escritora modifica el campo magnético de sectores específicos del plato.
3. Cada bit se representa por una orientación magnética específica.
4. Los datos se agrupan en sectores y pistas dentro de cilindros.

Organización lógica:

- Los archivos se dividen en bloques.
- El sistema de archivos mantiene un índice que indica dónde se ubica cada bloque.

Consulta:

¿Cuáles son los tipos de dato que existen en lenguajes de programación?

Tipos de dato en lenguajes de programación:

1. Numéricos:

- Enteros (int, long, short)
- Reales o de punto flotante (float, double, decimal) • Complementarios como unsigned (sin signo)

2. Caracteres y cadenas:

- carácter individual (char)

- cadenas de caracteres (string o varchar)

3. Booleanos:

- true / false

4. Otros tipos:

- enumeraciones (enum)

- tipos de datos compuestos (struct, clases)

- punteros (en lenguajes como C/C++)

¿Cuáles son los tipos de datos que existen en las bases de datos?

Tipos de dato en bases de datos:

1. Numéricos:

- Integer, smallint, bigint

- Decimal, numeric, float, real

2. Texto y caracteres:

- CHAR, VARCHAR, TEXT

3. Fechas y horas:

- DATE, TIME, DATETIME, TIMESTAMP

4. Otros:

- Booleano (dependiendo del sistema, puede ser BIT)

- Binarios (BLOB, BINARY)

¿Qué es un Collation en base de datos?

Un *Collation* (ordenamiento o intercalación) en una base de datos define las reglas para

comparar y ordenar cadenas de caracteres. Incluye aspectos como:

- La sensibilidad a mayúsculas y minúsculas (case sensitivity)
- La sensibilidad a acentos y diacríticos
- La forma de ordenar las cadenas (lexicográficamente o con reglas específicas)

GLOSARIO

➤ **Colas:** Es una forma de acceder los datos, contrario a la pila, se elimina (o desencola) por el frente y se ingresan los datos o se encolan por el final, se asemeja a las colas que observamos en el mundo real, en una cola en un banco, se atiende al primero que llega (el que está en el frente) y las personas ingresan al final de la cola.

➤ **Índice:** Es la posición de un elemento en la matriz, si el elemento está en la fila 2 y columna 1, decimos que la posición es (2,1), si el elemento está en la fila 1 columna 3 decimos que la posición es (1,3)

- **Lista doblemente enlazada:** Es un conjunto de nodos contiguos, que se les va asignando memoria a medida que se van creando, se enlazan entre ellos a través de dos punteros, Anterior: que apunta a la dirección del nodo anterior y Siguiente: que apunta al siguiente nodo.
- **Lista enlazada:** Es un conjunto de nodos, contiguos, a los que se les va asignando memoria a medida que se van creando, se enlazan entre ellos a través de un puntero que contiene la dirección al siguiente nodo. Cuando tiene un solo puntero se dice que es una lista enlazada simple.
- **Matriz:** Es una estructura que permite guardar un grupo de datos del mismo tipo en forma organizada. la matriz más sencilla tiene dos dimensiones conformadas por filas y columnas
- **Memoria:** Es un espacio de almacenamiento lógico para almacenar información en el computador durante determinado tiempo. La forma de representar esa información en memoria es a través del sistema binario de 1s y 0s.
- **Memoria dinámica:** Es cuando el tamaño, es decir, la cantidad de bytes que se requieren para almacenar un objeto (por ejemplo las listas enlazadas) es definido en tiempo de ejecución, a medida que el proceso necesite más espacio, va solicitando más memoria al sistema operativo
- **Memoria estática:** Es cuando el tamaño, es decir, la cantidad de bytes que se requieren para almacenar un objeto (por ejemplo un vector o una matriz) es definido en tiempo de compilación, por lo que en la ejecución del programa ya se conoce ese tamaño y no puede modificarse.
- **Nodo:** Es un elemento de una lista enlazada, está conformado por dos partes: Dato que es el espacio donde se guarda la información y Enlace que es el puntero que tiene la dirección al siguiente nodo.
- **Pila:** Es una estructura de datos que está clasificada como una lista lineal especial, porque la inserción (Apilar) y eliminación (Desapilar) de sus datos se realiza solo por la cima o tope.
- **LIFO:** (Último en entrar, primero en salir) se puede comparar con una lista de platos en el que el último que se enjabona, es el primero que se lava.

REFERENTES BIBLIOGRÁFICOS

- ✓ Joyanes, L., Fernández, M., Sánchez, L., Zanonero, I., (2005). Estructuras de datos en C. McGraw- Hill. ISBN 8448145127.
- ✓ Godoy, R., María, M., Cardona, A. (2006). Manejo Dinámico De Memoria En Un Programa De Elementos Finitos Orientado A Objetos. Mecánica Computacional Vol XXV, pp. 961-975. Santa Fe, Argentina.

Cibergrafía

- ✓ <https://ootips.org> sitio sobre programación orientada a objetos. Consultado en Octubre 10 de 2022.
- ✓ <https://learn.microsoft.com> librería especializada de herramientas de Microsoft. Consultado en Octubre 10 de 2022.
- ✓ <https://www.mongodb.com/docs> sitio oficial de MongoDB Inc. Consultado en Octubre 10 de 2022.

- ✓ <https://learn.microsoft.com/es-es/sql/sql-server> sitio oficial de la documentación para Microsoft SQL Server. Consultado en Octubre 10 de 2022.
- ✓ <https://docs.microsoft.com/en-us/dotnet/standard/automatic-memory-management> sitio oficial de la documentación para Microsoft SQL Server. Consultado en Octubre 10 de 2022.

CONTROL DEL DOCUMENTO

	Nombre	Cargo	Dependencia	Fecha
Autor (es)	Luis Fernando Sánchez Pérez.	Instructor	CTMA/TICS- Electrónica.	20/4/2025

CONTROL DE CAMBIOS (diligenciar únicamente si realiza ajustes a la guía)

	Nombre	Cargo	Dependencia	Fecha	Razón del Cambio
Autor (es)	Luis Fernando Sánchez Pérez	Instructor	CTMA/TICS- Electrónica.	9/5/2025	Modificación de Actividades de Apropiación del conocimiento